Goodput-Aware Adaptive Speculative Decoding for Batched LLM Serving: Midterm Report

Vaishnavi Athrey Ramesh
Carnegie Mellon University
vathreyr@andrew.cmu.edu

Aritra Raychaudhuri
Carnegie Mellon University
araychau@andrew.cmu.edu

Abstract

Speculative decoding accelerates large language model inference by letting a small drafter propose a block of gamma tokens that the large target model verifies in a single forward pass. The speedup depends on the speculation length gamma, which current serving systems such as vLLM expose as a static flag. We argue that the best gamma is load dependent and should be chosen online. This report presents four pieces of midterm progress. First, a roofline analysis that predicts how the throughput optimal gamma varies with batch size and acceptance rate. Second, an adaptive controller that estimates the per token acceptance rate online and selects gamma by maximizing a throughput proxy, together with a corrected acceptance estimator that fixes a normalization bias in our initial version. Third, a discrete event serving simulator with an end to end latency metric that includes queue wait, and a policy comparison averaged over five seeds against five static speculation lengths, the no speculation baseline, and a reference oracle. Fourth, an initial hardware study on an NVIDIA V100. In simulation the throughput optimal gamma falls from 4 at batch size 1 to 1 at large batch, and the adaptive controller matches the best policy at low load and is at or above every tested static gamma from the saturation point onward while tracking the oracle. An ablation isolates the contribution of online acceptance estimation from batch awareness and shows that, in our drift workload, batch awareness provides most of the benefit.

1 Introduction

A language model decodes one token per forward pass of a large target model, and that pass is dominated by reading the weights and the key value cache from memory. Speculative decoding (Leviathan et al. [1], Chen et al. [2]) reduces the number of target passes by having a cheap drafter propose gamma candidate tokens that the target verifies in parallel, emitting several tokens per pass when the drafter is accurate.

The speedup is governed by the speculation length gamma and the per token acceptance rate alpha. A larger gamma emits more tokens per step when acceptance is high, but it also forces the target to verify batch times (gamma plus 1) token positions, which wastes compute whenever those extra tokens are rejected. The right value therefore depends on how busy the GPU is. At low load the target decode is memory bound and the GPU has idle compute, so verifying extra speculative tokens is almost free. At high load the target decode becomes compute bound and the wasted verification competes with useful work, which lowers throughput and inflates the per output token latency that serving SLOs are written against. Despite this, production stacks fix gamma as a static flag.

Our idea is to treat gamma as an online scheduling decision. The controller selects gamma to maximize a throughput proxy, the expected accepted tokens per unit of GPU time at the current batch size. We then evaluate the resulting policy on goodput, defined separately as the number of requests per second whose end to end latency meets a per output token SLO. The throughput proxy and goodput are related but not identical, and we keep them distinct throughout. This report covers the design, the roofline analysis that motivated it, the simulation study with the corrected estimator and seed averaging, and our initial hardware progress.

2 Related Work

Leviathan et al. [1] and Chen et al. [2] introduced speculative decoding and the rejection sampling scheme that makes the accepted output distribution identical to standard decoding, and give the expected tokens per step as a function of alpha and gamma that we reuse in our objective. EAGLE (Li et al. [3]) trains a lightweight drafter head on the target features to raise acceptance, and EAGLE-3 (Li et al. [7]) scales that approach with a training time test. We adopt EAGLE-3 as a high acceptance comparison in the final phase. vLLM (Kwon et al. [4]) is the serving system we target, with continuous batching and speculative decoding, although the speculation length is set once at engine construction. The roofline model (Williams et al. [5]) is the performance model we use to reason about the memory bound versus compute bound regimes.

The closest prior work is SmartSpec (Liu et al. [6]), which also argues that the speculation length should be chosen from a goodput metric and dynamically sets it per request from the historical acceptance rate and a linear batch execution time model. Our project shares this premise and should be read as a reproduction and extension of that idea rather than a new one. We differ in three ways pursued in the final phase. We ground the choice in an explicit roofline cost model calibrated from microbenchmarks, we study the policy across two GPUs and two drafters including the extremely memory bound regime, and we compare the cost model controller against a model free throughput feedback controller. The midterm scope is the simulation study and the first hardware measurements.

3 Methodology

3.1 Roofline cost model

A single target pass over n positions costs t_target(n) equals t_mem plus t_flop times n, where t_mem is the memory bound floor of a decode step and t_flop is the marginal compute per token. For a batch of B at speculation length gamma, the macro step cost is t_macro(B, gamma) equals gamma times t_draft(B) plus t_target(B times (gamma plus 1)). Following Leviathan et al. [1], the expected accepted tokens per request per step is E[tokens given alpha, gamma] equals (1 minus alpha to the power gamma plus 1) divided by (1 minus alpha). The controller maximizes a throughput proxy, accepted tokens per unit of GPU time, gamma* equals argmax over gamma of E[tokens given alpha, gamma] divided by t_macro(B, gamma). Because t_macro grows with B times (gamma plus 1) times t_flop, the denominator penalizes long speculation exactly when the batch is large, so the optimal gamma should shrink as B grows and grow as alpha grows.

3.2 Acceptance estimator, and a correction

The controller needs an estimate of the per token acceptance probability alpha. Our initial version fed it the accepted prefix fraction, accepted tokens divided by gamma times batch. This is biased low. Under leading run acceptance the run stops at the first rejection, so positions after it are never tested, and the expected accepted prefix fraction is therefore below alpha, about 0.47 at a true alpha of 0.72 and gamma of 4, which pushes the controller toward shorter speculation. We corrected the denominator to the number of observed trials, meaning draft positions that were actually resolved, which is every accepted token plus the single rejection that terminated each run, capped at gamma when the whole block is accepted. We call this an observed trial estimator. It addresses the normalization bias, but we do not claim it is unbiased in finite samples. For gamma equal to 2 on a single sequence its expectation is alpha plus alpha times (1 minus alpha) divided by 2, and pooling trials across a heterogeneous batch weights requests by how many trials they produced. All results in this report use the observed trial estimator.

3.3 Adaptive controller and reference oracle

The controller keeps an exponential moving average of the observed trial alpha estimate and at each step selects gamma* using the objective in Section 3.1. It needs only the current running batch size, the online alpha estimate, and the cost model, all of which a serving scheduler already has. We also run a reference oracle that selects gamma using the request true current acceptance rate rather than an online estimate. It is not realizable in practice and is not a proven upper bound on long run goodput. It is a reference point for how much the online estimate costs the controller.

3.4 Ablation baseline

To separate the benefit of online acceptance estimation from the benefit of batch awareness, we add a batch aware policy that uses a fixed acceptance rate. It picks gamma from the current batch size through the same cost model objective but freezes alpha at the undrifted value and never updates it. The gap between the full adaptive controller and this policy isolates the contribution of the online estimate, which the static gamma baselines cannot.

3.5 Serving simulator and metrics

The discrete event simulator models Poisson arrivals at rate lambda, per request output lengths, a maximum batch cap with a queue when overloaded, heterogeneous per request acceptance with optional within request drift, and leading run accept sampling. All timing comes from the cost model and all policy decisions come from the controller, so the same controller object can later move into vLLM unchanged. We report throughput in tokens per second and an end to end normalized latency equal to the time from arrival to completion divided by the number of output tokens, which includes queue wait. Goodput is the number of requests per second whose end to end normalized latency meets the SLO. We use the end to end metric, rather than decode only latency, because the latter excludes queueing and does not degrade under overload. All policy comparisons are averaged over five seeds and reported with standard deviations so that small differences are not read as signal.

4 Experimental Configuration

Table 1 lists the configuration. The cost model constants are illustrative order of magnitude values, to be calibrated from microbenchmarks in the hardware phase.

Table 1. Simulation configuration.

Setting	Value
Requests per run	1500
Mean output length	220 tokens, lognormal, clipped 16 to 1024
Per request acceptance	mean 0.72, spread 0.12, clipped 0.35 to 0.95
Within request drift (E3 only)	0.0012 per token decay
Maximum batch	64
Speculation lengths considered by controller	1 to 8
Static baselines evaluated	gamma 2, 3, 4, 5, 6, plus no speculation
Controller EMA decay, initial alpha	0.9, 0.7
SLO on end to end latency	0.045 s per output token
Arrival rates lambda	5, 10, 20, 35, 50, 70, 90, 120, 150 req/s

Setting	Value
Seeds	7, 11, 17, 23, 29
Cost model	t_mem 12 ms, t_flop 0.11 ms/token, draft 1.3 ms and 0.012 ms/token

5 Results

All numbers are outputs of the simulator with the observed trial estimator, averaged over five seeds, with standard deviations shown.

5.1 Load versus best speculation length

Table 2 and Figure 1 are generated from the same cost model sweep. The throughput optimal static gamma falls from 4 at small batch to 1 at large batch, which matches the roofline prediction that long speculation stops paying off once the verification pass becomes compute bound. Each point in Figure 1 is annotated with its gamma value to match the table exactly.

Table 2. Throughput optimal static gamma versus batch size (cost model, alpha equals 0.72), identical to the annotated curve in Figure 1.

Batch size	1	2	4	8	16	32	48	64	96	128
Best gamma	4	4	4	3	3	2	2	1	1	1

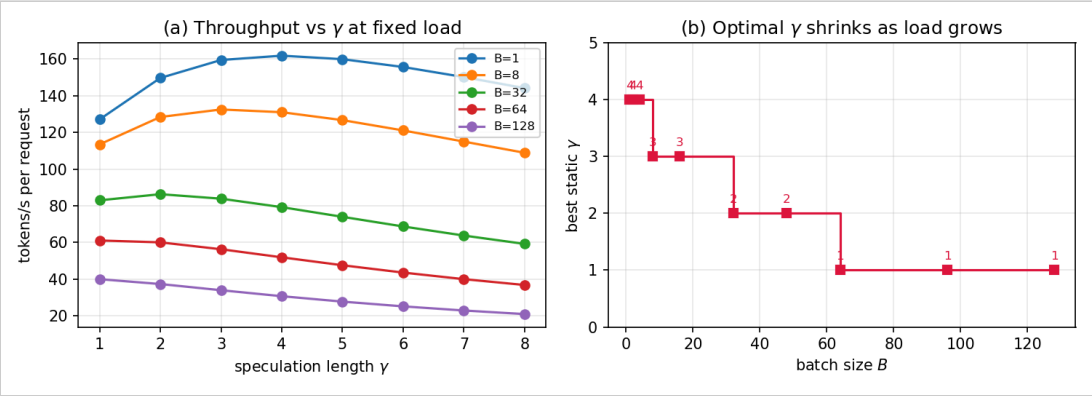


Figure 1. Tokens per second versus gamma at several batch sizes (left) and the throughput optimal static gamma versus batch size (right, each point labeled). The optimum shrinks from 4 to 1 as load grows.

5.2 Policy comparison across load

Figure 2 compares the policies across an arrival rate sweep on throughput and end to end goodput, with error bars showing plus or minus one standard deviation over the five seeds. At low load, lambda of 5 and 10, the system is unsaturated and every policy attains essentially the same goodput, about 5.0 and 9.9 requests per second, so there is nothing to adapt. The policies separate once the system saturates. At the saturation knee, lambda of 20, the adaptive controller reaches 5.90 plus or minus 0.40 requests per second against 5.28 plus or minus 0.39 for the best static setting, static gamma 2, and matches the oracle at 5.91 plus or minus 0.41. From the knee onward the adaptive controller stays at or above every tested static gamma, for example 2.15 plus or minus 0.10 against 2.05 plus or minus 0.11 for static gamma 2 at lambda 50, and 1.70 plus or minus 0.08 against 1.63 plus or minus 0.07 at lambda 150, while the longer static settings and no speculation fall well behind, for example static gamma 6 at 0.74 and no speculation at 1.16 at lambda 150. We therefore state the claim precisely. The adaptive controller is competitive at low load and at or above the

tested static baselines from the saturation point onward, tracking the oracle across the sweep. We do not claim a uniform win at every load, because at low load the policies are tied within noise.

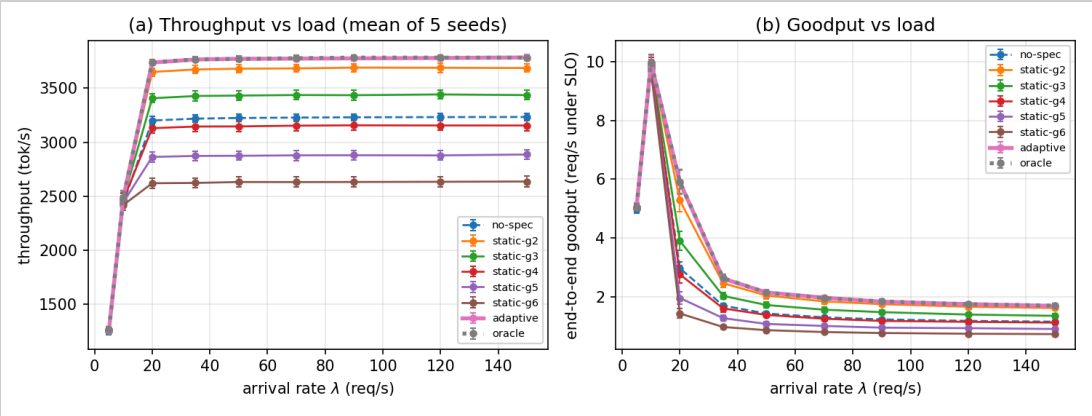


Figure 2. Throughput (left) and end to end goodput (right) versus arrival rate, mean of five seeds with plus or minus one standard deviation error bars. The adaptive controller tracks the oracle and stays at or above every static baseline once the system saturates.

5.3 Drift ablation

Table 3 reports the drift workload, where per request acceptance decays as a generation gets longer. Batch awareness alone, the batch only policy with a fixed acceptance rate, already lifts goodput from 1.50 plus or minus 0.08 for the best static setting to 2.03 plus or minus 0.11, because it shortens speculation as the batch grows. Adding online acceptance estimation drives the mean speculation length lower, 1.02 against 1.21, which is the expected signature of tracking the decaying acceptance. The resulting goodput change, 2.03 to 2.06, and the throughput change, 3231 plus or minus 35 to 3279 plus or minus 25 tokens per second, are within about one standard deviation, so we report the online acceptance gain as preliminary rather than established. The clear and reproducible effect is the lower mean gamma, and quantifying the goodput gain precisely needs more seeds and a workload with stronger drift, which we plan for the final report.

Table 3. Drift workload, mean plus or minus one standard deviation over five seeds. Goodput is end to end requests per second meeting the SLO.

Policy	Throughput (tok/s)	Goodput (req/s)	Mean gamma
static gamma 2	2907 plus/minus 27	1.50 plus/minus 0.08	2.00
static gamma 5	1945 plus/minus 26	0.49 plus/minus 0.03	5.00
batch only, fixed alpha	3231 plus/minus 35	2.03 plus/minus 0.11	1.21
adaptive, online alpha	3279 plus/minus 25	2.06 plus/minus 0.11	1.02

6 Hardware Progress

We have completed an initial hardware study on an NVIDIA V100-SXM2-32GB. We ran vLLM 0.11.0 with n-gram speculative decoding on Qwen2.5-7B-Instruct, using the Triton attention backend and float16 since Volta does not support FlashAttention. We calibrated the cost model constants from decode microbenchmarks, obtaining t_mem of 22.9 ms and t_flop of 0.145 ms per token with a fit R squared of 0.94, which confirms a strongly memory bound decode. We then measured a speculation length by batch size grid and compared policies by selecting, for each batch size, the gamma the controller would choose given the measured acceptance, which reproduces on hardware the prediction that the optimal gamma shifts with load. We are precise about scope. This comparison selects results from a measured grid and does not demonstrate live per step adaptation

inside a running vLLM server. In vLLM 0.11.0 the speculation length is fixed at engine construction, so true per step control requires patching the proposer, which remains in progress. The final report will present the full hardware results, including an EAGLE-3 comparison and a throughput feedback controller.

7 Discussion and Limitations

The simulation supports the expected behavior under the simulated configurations with the observed trial estimator, and shows that adaptation helps once the system saturates. Two limitations shape how we read the numbers. First, the simulator is a coarse model whose absolute values depend on illustrative cost constants, so we treat the comparisons qualitatively and defer absolute claims to hardware. Second, although the end to end metric now includes queue wait, a full evaluation of tail latency under real traces belongs to the hardware phase. The open hardware question is whether a real decode step is so memory bound that a static gamma is already close to optimal across the batch range, in which case the benefit of adaptation would be robustness and the absence of manual tuning rather than a large speedup. Our V100 calibration suggests the memory bound regime is real, which we will study directly.

8 Timeline

Completed by this checkpoint, as of 4 November 2026. Roofline cost model and the closed form optimal gamma analysis, the adaptive controller, the reference oracle, the batch only ablation, the observed trial estimator correction, the simulator with the end to end metric, the E1 to E3 experiments with five seed averaging, and the initial V100 calibration and grid.

Planned milestones.

Window	Dates	Deliverable
Week 1	Nov 10 to Nov 16	Patch the vLLM proposer for live per step gamma control, validate on the V100 against the offline grid
Week 2	Nov 17 to Nov 23	A100 calibration and experiments with n-gram and EAGLE-3, build the throughput feedback controller
Week 3	Nov 24 to Nov 30	Serving goodput sweep over a request trace with tail latency percentiles, direct comparison against SmartSpec
Final	Dec 1 to presentation	Final analysis, figures, report, and presentation

9 Conclusion

We framed the speculation length as an online scheduling decision and predicted from a roofline model that the throughput optimal gamma shrinks with batch size and grows with acceptance. With the corrected estimator and five seed averaging, the controller matches the best policy at low load and is at or above every tested static gamma once the system saturates, an ablation attributes most of the benefit to batch awareness with the online acceptance gain preliminary, and an initial V100 study reproduces the central prediction on hardware. The remaining work is live vLLM integration, the A100 and EAGLE-3 experiments, the throughput feedback controller, and a comparison with SmartSpec.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In Proc. ICML, 2023.

[2] C. Chen, S. Borgeaud, G. Irving, et al. Accelerating large language model decoding with speculative sampling. arXiv:2302.01318, 2023.

[3] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In Proc. ICML, 2024.

[4] W. Kwon, Z. Li, S. Zhuang, et al. Efficient memory management for large language model serving with PagedAttention. In Proc. SOSP, 2023.

[5] S. Williams, A. Waterman, and D. Patterson. Roofline: An insightful visual performance model for multicore architectures. Communications of the ACM, 52(4):65 to 76, 2009.

[6] X. Liu, C. Daniel, L. Hu, W. Kwon, Z. Li, X. Mo, A. Cheung, Z. Deng, I. Stoica, and H. Zhang. Optimizing speculative decoding for serving large language models using goodput. arXiv:2406.14066, 2024.

[7] Y. Li, F. Wei, C. Zhang, and H. Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. arXiv:2503.01840, 2025.

Team Contributions

Member	Responsibilities
Vaishnavi Athrey Ramesh	Roofline cost model, adaptive controller and estimator correction, initial V100 hardware study
Aritra Raychaudhuri	Serving simulator and end to end metric, ablation baseline, seed averaged experiments and report